

Name:- Krishna Nagare

PRN:- 2124UCSM1032.

Cloud Security – Project-Based Learning

IAM & Least Privilege Enforcement (Unit 2)

This Terraform project builds a least-privilege IAM structure in AWS.

❖ What the project creates

- **IAM Groups:** Admins, Developers, and Auditors, each with a custom policy tailored to the permissions required for that role.
- **MFA Enforcement:** An additional policy attached to the Admins group denies most AWS actions unless the user has authenticated with MFA (aws:MultiFactorAuthPresent). Users can still manage their own MFA devices so they are not locked out.
- **EC2 → S3 Access Without Hardcoded Credentials:** A custom IAM role (EC2-S3-Access-Role) and instance profile allow an EC2 instance to access a specific S3 bucket without storing AWS access keys on the instance.

❖ Files

File	Purpose
main.tf	Defines all IAM groups, policies, the EC2 IAM role, and instance profile
variables.tf	Defines input variables such as AWS region and S3 bucket name
outputs.tf	Provides useful outputs after deployment
terraform.tfvars.example	Example variables file; copy it to terraform.tfvars and provide your own bucket name

❖ How the Least-Privilege Design Works

Admins

The **Admins** group receives broad operational permissions for:

- EC2
- S3
- IAM group and role management
- CloudWatch and CloudWatch Logs

However, administrative access is intentionally restricted. The policy explicitly denies access to:

- Billing
- AWS Organizations
- Other sensitive account-level actions

Therefore, the Admins role is **not equivalent to the AWS root user**.

Developers

The **Developers** group can:

- Run, stop, and terminate EC2 instances
- Read and write objects in the application S3 bucket

IAM management permissions are explicitly denied, preventing developers from modifying IAM users, groups, roles, or policies.

Auditors

The **Auditors** group has read-only access across the services required for auditing:

- IAM
- S3
- EC2
- CloudTrail
- AWS Config

Permissions are limited to read-only actions such as Get*, List*, and Describe*, so auditors can inspect resources without modifying them.

EC2 → S3 IAM Role

The EC2-S3-Access-Role can be assumed by EC2 through an instance profile.

Its permissions are restricted to the specified application S3 bucket and its objects rather than using "Resource": "*".

This allows an EC2 instance to access the required S3 resources **without storing AWS access keys or secret credentials on the instance**.

Deploy It Yourself

Clone the repository and navigate to the project directory:

```
git clone <your-repo-url>
```

```
cd iam-least-privilege-project
```

Copy the example variables file:

```
cp terraform.tfvars.example terraform.tfvars
```

Edit terraform.tfvars and provide your AWS region and a globally unique S3 bucket name.

Initialize Terraform:

```
terraform init
```

Review the deployment plan:

```
terraform plan
```

Apply the configuration:

```
terraform apply
```

You will need AWS credentials configured locally using either:

```
aws configure
```

or environment variables such as `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`.

The project is intended to be deployed in **your own AWS account or sandbox environment** so that you can obtain real AWS Console output and screenshots for the assignment.

❖ Getting Your Screenshot

After terraform apply completes successfully, you can capture screenshots of the following:

1. IAM Groups

Go to:

AWS Console → IAM → User groups

Show:

- Admins
- Developers
- Auditors
- Their attached policies

2. EC2 IAM Role

Go to:

AWS Console → IAM → Roles → EC2-S3-Access-Role

Show:

- The role name
- The EC2 trust relationship (ec2.amazonaws.com)
- The attached permissions policy

3. Terraform Deployment

A terminal screenshot showing:

terraform apply

completed successfully and the resources were created.

4. Optional MFA Verification

To demonstrate MFA enforcement, an IAM user belonging to the Admins group can attempt an AWS action without MFA.

The request should be denied with an **AccessDenied** error.

After MFA authentication, permitted actions should be allowed according to the group's policies.

❖ Cleanup

When you are finished testing, remove the resources created by Terraform:

```
terraform destroy
```

This helps prevent unnecessary AWS charges and removes the IAM resources created by the project.